Building in Jenkins using Azure SOP

Cognizant

Table of Contents

1. Building in Jenkins using Azure....3
1.1 Description....3
1.2 Architecture Diagram....3
1.3 Lab Steps....4
1.4 Troubleshooting....5
1.5 Supporting References....5

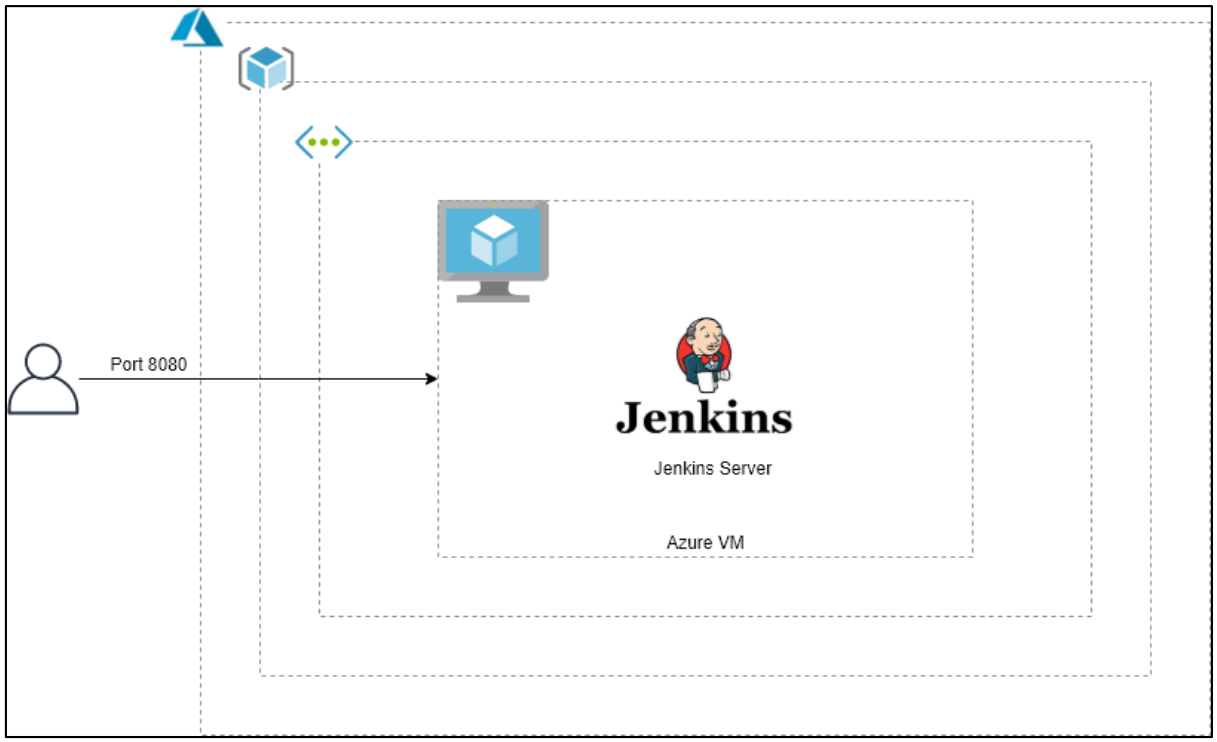
1. Building in Jenkins using Azure

1.1 Description

Jenkins offers a simple way to set up a continuous integration or continuous delivery (CI/CD) environment for almost any combination of languages and source code repositories using pipelines, as well as automating other routine development tasks. With Jenkins, organizations can accelerate the software development process by automating it.
In this lab activity you will be creating an Azure VM where you will work on installation and setup of Jenkins server. When you are finished, you have a Jenkins server running in Azure building a sample Java app.

1.2 Architecture Diagram

The diagram below displays a visual representation of the application architecture:



1.3 Lab Steps

Follow the steps outlined below to achieve the objective of this lab exercise:

1. Log In to the Azure Portal and go to your respective resourcegroup with your empid.

2. Launch a Ubuntu Virtual Machine:
   a. Inside your RG launch an Ubuntu VM with least size available.
   b. Once VM is deployed, connect to it.
   c. Remember to add port 8080 to your NSG on which Jenkins server will work.

3. Install Jenkins Server:
   a. Jenkins is a Java application, the first step is to install Java. Update the package index and install the Java 8 OpenJDK package with the following commands:

   ```
   sudo apt update
   sudo apt install openjdk-8-jdk
   ```

   b. Import the GPG keys of the Jenkins repository using the following wget command:

   ```
   wget -q -O - https://pkg.jenkins.io/debian/jenkins.io.key | sudo
   apt-key add -
   ```

   c. The above command will return OK, and add the Jenkins repository to the system with the following command:

   ```
   sudo sh -c 'echo deb http://pkg.jenkins.io/debian-stable binary/
   > /etc/apt/sources.list.d/jenkins.list'
   ```

   d. Once the Jenkins repository is enabled, update the apt package list and install the latest version of Jenkins:

   ```
   sudo apt update
   sudo apt install jenkins
   ```

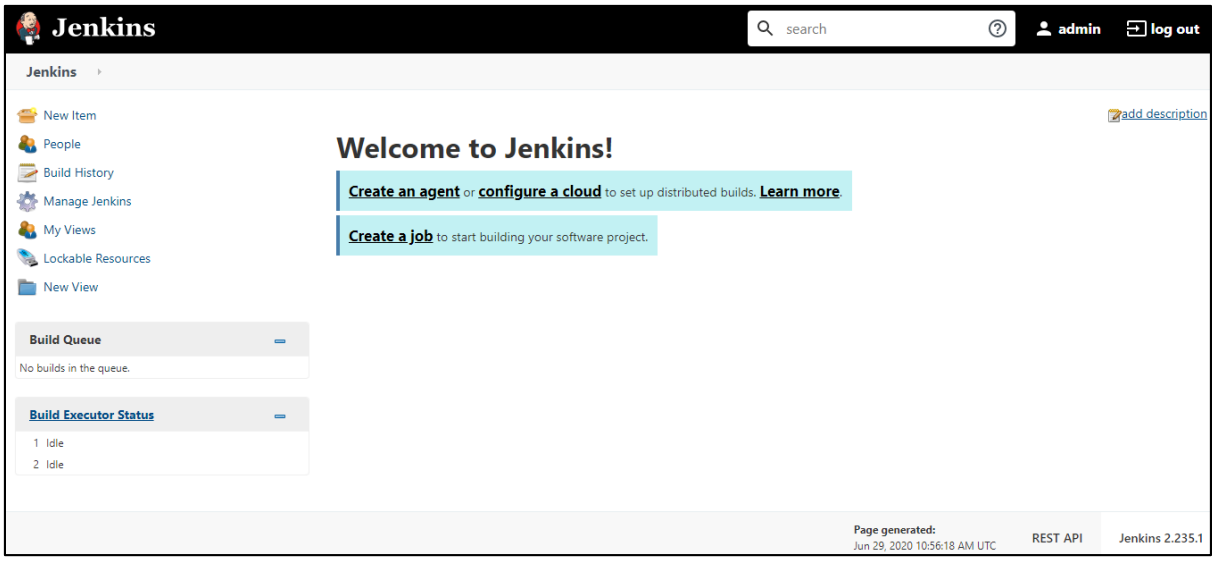
   Note: Jenkins service will automatically start after the installation process is complete. You can verify it by printing the service status:

   ```
   systemctl status jenkins
   ```

4. Connect to Jenkins:
   a. Use the Public Ip Address of the VM with the port number i.e. 8080 to access Jenkins. E.g. 52.66.21.210:8080
   b. Unlock the Jenkins dashboard for the first time using this initial password. Enter the following command to access the admin password to start Jenkins:

   ```
   sudo cat /var/lib/jenkins/secrets/initialAdminPassword
   ```

c. Select Install Suggested Plugins and you should be able to find Jenkins dashboard.



5. Create first job and Build the code:
   a. Select **Create new jobs** from the Jenkins console, then name it **mySampleApp** and select **Freestyle project** > **OK**.
   b. Select the **Source Code Management** tab, enable **Git**, and enter the following URL in **Repository URL** field: https://github.com/spring-guides/gs-spring-boot.git.
   c. Select the **Build** tab, then select **Add build step**, **Invoke Gradle script**. Select **Use Gradle Wrapper**, then enter complete in **Wrapper location** and build for **Tasks**.
   d. Select **Advanced** and then enter complete in the **Root Build script** field. Select **Save**.
   e. Select **Build Now** to compile the code and package the sample app. When your build completes, select the **Workspace** link for the project.
   f. Navigate to **complete/build/libs** and ensure the **gs-spring-boot-0.1.0.jar** is there to verify that your build was successful. Your Jenkins server is now ready to build your own projects.

1.4 Troubleshooting

S. No	Problem	Solution
1	sub-process /usr/bin/dpkg returned an error code (1) while installing jenkins	Make sure java is installed on VM
2	Jenkins Dashboard is not displayed	Make sure all the installations are done and port 8080 is added to NSG

1.5 Supporting References

Refer the below link for additional information:

1. https://docs.microsoft.com/en-us/azure/virtual-machines/linux/quick-create-portal